Koneru Lakshmaiah Education Foundation
(Category -1, Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Approved by AICTE ISO 21001:2018 Certified
Campus: Green Fields, Vaddeswaram - 522 302, Guntur District, Andhra Pradesh, INDIA.
Phone No. +91 8645 - 350 200; www.klef.ac.in; www.klef.edu.in; www.kluniversity.in
Admin Off: 29-36-38, Museum Road, Governorpet, Vijayawada - 520 002. Ph: +91 - 866 - 3500122, 2576129

NETWORK BANDWIDTH ALLOCATION SYSTEM

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

By

ANKAMREDDI EASWARA SRI SAI VENKAT
Student ID: 2601050119

under the supervision of

DR. RAJU ANITHA
Professor

Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation (KLEF)
Vaddeswaram, Andhra Pradesh, India

2026

DECLARATION

I hereby declare that the project report entitled “Network Bandwidth Allocation System” is prepared for academic purposes under the guidance of Prof. RAJU ANITHA, Professor, Department of Computer Science and Engineering. The report describes the design and implementation of a system that demonstrates network maximum-flow computation, dynamic-programming-based bandwidth allocation, and parallel benchmarking.

I declare that the technical descriptions, algorithms, implementation details, and evaluation discussion have been prepared to represent the project accurately. The implementation is based on Python, Flask, a maximum-flow solver, a 0/1 knapsack allocation solver, and a multiprocessing benchmark. Sample performance figures shown in the report are identified as observations from the supplied dashboard screenshots and should be interpreted in the context of the machine and inputs used.

Sources consulted in preparing the report are acknowledged in the references. Any tests or deployment steps that have not been executed independently are presented as recommended validation rather than as completed experimental results. This report is submitted in accordance with the institution’s academic-integrity requirements.

Place: Vaddeswaram

Date: 13-10-2026

Signature of the Student: ANKAMREDDI EASWARA SRI SAI VENKAT (2601050119)

INDEX

S. No.	Topics	Page No.
1	**Introduction and Problem Definition**	1 – 3
2	**Literature Review and Theoretical Foundations**	3 – 6
3	**Proposed Methodology and System Architecture**	6 – 12
4	**Implementation and API Design**	13 – 16
5	**Experimental Setup, Results and Discussion**	16 – 21
6	**Conclusion, Limitations and Future Scope**	21 – 22
7	**References**	22
8	**Appendices and User Guide**	23 – 24

LIST OF TABLES

No.	Table Name	Page
2.1	Comparison of related approaches	5
3.1	Software requirements	7
3.2	System architecture layers	8
3.3	Complexity of the implemented modules	12
4.1	Project file organization	13
4.2	Worked allocation example (budget = 10 units)	14
4.3	API endpoints and request fields	15
4.4	Sample network links and capacities	15
5.1	Test environment and parameters	17
5.2	Maximum-flow dashboard observations	18
5.3	Allocation dashboard observations	18
5.4	Parallel benchmark dashboard observations	19
5.5	Test plan and expected observations	20
8.1	Glossary of terms	24

ABSTRACT

Modern computer networks carry traffic from a range of applications, including video delivery, voice communication, web services, backup operations, gaming, and Internet of Things devices. Network capacity is limited, and a poor allocation strategy can leave some links congested while other capacity remains underused. A practical bandwidth-management tool should estimate the maximum throughput supported by a network, select traffic demands under a capacity budget, and provide measurable evidence about the performance of its computational methods.

This project presents a Network Bandwidth Allocation System that combines three techniques: maximum-flow routing, dynamic programming, and parallel benchmarking. The maximum-flow module models a network as a directed graph and applies the Edmonds–Karp algorithm to determine the largest feasible flow from source S to sink T. The solver reports edge-level flow values and a minimum cut that helps explain the capacity bottleneck. The allocation module formulates traffic selection as a 0/1 knapsack problem, using dynamic programming to choose a subset of flows that maximizes total utility without exceeding a configured bandwidth budget.

A Flask application exposes the computations through JSON endpoints and serves a browser dashboard with separate views for routing, allocation, and benchmarking. The benchmark evaluates deterministic synthetic allocation scenarios serially and through a multiprocessing pool, then reports elapsed time, worker count, speedup, and whether the results match. The supplied dashboard examples show a maximum throughput of 18 Mbps, an allocation value of 240 under an 80 Mbps budget, and a sample parallel speedup of 1.59× across 400 scenarios.

The system is intended as an educational prototype for understanding graph algorithms, optimization under constraints, API integration, and parallel computation. Its synthetic scenarios provide repeatable demonstrations but do not represent production network telemetry. Further work can introduce live measurements, quality-of-service policies, fairness constraints, stronger request validation, and controlled integration with network-management systems.

Keywords: Network bandwidth, maximum flow, Edmonds–Karp, minimum cut, dynamic programming, 0/1 knapsack, Flask, multiprocessing, traffic engineering.

1 INTRODUCTION

1.1 Background

Computer networks support applications whose traffic demands differ in size, priority, timing, and sensitivity to delay. Video services may require sustained throughput, voice communication benefits from predictable delivery, backup jobs can often tolerate delay, and web requests may arrive in bursts. As the number of connected systems grows, efficient use of available bandwidth becomes an important network-management problem.

A network can be represented as a directed graph in which vertices represent routers, gateways, or logical network locations and edges represent communication links. Each edge has a capacity that limits the amount of traffic it can carry. The maximum-flow problem asks how much traffic can be transmitted from a source to a sink while respecting all link capacities and maintaining flow conservation at intermediate vertices.

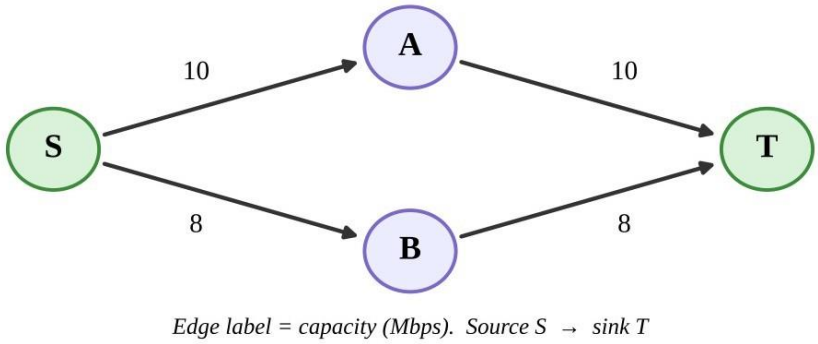


Figure 1.1: Directed network model with source, intermediate nodes and sink

Bandwidth allocation is related but distinct. Instead of finding routes through a graph, an allocation policy chooses which candidate traffic demands should be admitted under a limited total capacity. If each candidate has a demand and a utility score, the 0/1 knapsack model provides a clear optimization formulation. Dynamic programming can examine feasible capacity states and identify a combination with the highest total value.

This project combines these ideas in a small web-based application. It is designed to make the algorithms visible and testable through a dashboard rather than treating the solver as an isolated command-line program.

1.2 Problem Definition

Static or informal bandwidth allocation can make poor decisions when the network is constrained. A route may appear available but be limited by a smaller-capacity link later in the path. Similarly, accepting traffic requests in arrival order may use capacity on low-value demands and leave insufficient room for a more valuable combination. Performance claims about optimization code also require measurement, because parallel processing can introduce overhead that outweighs its benefit for small workloads.

The project addresses three connected problems. First, it needs to calculate the maximum feasible throughput of a directed network and identify the links forming a minimum cut. Second, it needs to select a set of traffic flows that maximizes total value without exceeding a bandwidth limit. Third, it needs to compare serial and parallel execution on repeatable scenarios and verify that both modes return equivalent results.

1.3 Project Objectives

- Represent a network as directed edges with capacities and calculate maximum flow from S to T.
- Report the flow assigned to each original edge and identify the minimum-cut edges.
- Implement a 0/1 knapsack dynamic-programming solver for flow selection.
- Provide a Flask API and browser interface for entering inputs and viewing results.
- Generate deterministic synthetic scenarios for serial and multiprocessing evaluation.
- Measure elapsed time and speedup while checking that serial and parallel outputs match.

1.4 Scope

The current implementation focuses on algorithmic demonstrations with user-provided or synthetic data. It does not collect live traffic from routers, reserve bandwidth on physical network devices, or enforce service-level agreements. Capacity values and flow utility scores are treated as input data. The dashboard presents optimization results for analysis and does not automatically apply changes to a real network.

1.5 Existing System

In a basic network environment, bandwidth allocation may be performed through fixed priorities, static routing, or manual capacity planning. These methods are simple to understand but may not explain the global bottleneck or identify the best combination of competing demands. A greedy allocation policy can be fast, yet it does not always find the maximum total utility when flow demands and values differ.

Separate command-line scripts can calculate maximum flow or solve a knapsack problem, but they may not provide a unified interface for experimentation. Benchmarking performed on different input sets can also make comparisons unreliable. A single application that exposes the methods through structured endpoints allows users to repeat scenarios and compare outputs consistently.

1.6 Proposed System

The proposed system combines a graph solver, a capacity-constrained allocation solver, a Flask service layer, and a dashboard. The max-flow module uses breadth-first search to find augmenting paths in a residual graph. The allocation module fills a dynamic-programming table over flows and capacity states, then reconstructs the selected subset. The benchmark module evaluates seeded scenarios serially and in a multiprocessing pool.

The three modules complement one another. Maximum flow estimates the network's theoretical throughput under the supplied graph, allocation determines which candidate demands should be accepted under a single budget, and benchmarking measures how the

allocation computation behaves under repeated workloads. The modules are separate so that each algorithm can be tested and understood independently.

1.7 Advantages of the Proposed System

- Provides a visual way to explore maximum-flow and minimum-cut results.
- Shows edge-level flow assignments rather than only a total throughput number.
- Uses dynamic programming to find an optimal subset for the stated 0/1 allocation model.
- Separates computation from the dashboard through JSON API endpoints.
- Uses seeded synthetic scenarios to make serial and parallel results comparable.
- Reports benchmark measurements and result equality together.
- Can be extended with priorities, fairness policies, telemetry, or multi-region routing.

1.8 Applications

The concepts demonstrated by the project are relevant to network capacity planning, traffic engineering, service admission control, cloud resource allocation, data-center communication, and educational demonstrations of graph algorithms. The max-flow formulation can help identify structural bottlenecks in a simplified topology, while the knapsack model can explain trade-offs between demand and utility.

These applications are conceptual rather than claims of direct production deployment. Real network management must account for delay, packet loss, redundancy, dynamic traffic, operational policies, and device configuration. The current system is a foundation for experimentation and algorithm comparison.

2 LITERATURE REVIEW AND THEORETICAL FOUNDATIONS

2.1 Introduction

Bandwidth optimization connects several established areas of computer science: graph theory, combinatorial optimization, dynamic programming, web service design, and parallel computing. Maximum-flow algorithms provide a formal way to reason about throughput in a capacitated graph. Knapsack algorithms model admission decisions when each request consumes a resource and contributes a value. Parallel processing provides a way to evaluate many independent scenarios when the workload is large enough to justify process overhead [3], [4], [8], [14].

The purpose of this chapter is to establish the concepts used by the implementation and explain why the chosen methods are suitable for a small, inspectable prototype. The project uses well-known algorithmic approaches rather than proposing a new optimization theorem. Its contribution is the integration of the methods into a dashboard with repeatable experiments.

2.2 Directed Graph Representation

A directed network is represented as $G = (V, E)$, where V is the set of vertices and E is the set of directed edges. Each edge (u, v) has a non-negative capacity $c(u, v)$. The source s

represents the origin of traffic and the sink t represents its destination. A flow assigns an amount f(u, v) to each edge, subject to capacity and conservation constraints.

The capacity constraint is $0 \leq f(u,v) \leq c(u,v)$. For each intermediate vertex, total incoming flow must equal total outgoing flow. The source emits the net flow and the sink receives the same amount. The objective is to maximize the total flow reaching the sink. This abstraction does not model every real-network detail, but it gives a precise baseline for capacity analysis.

2.3 Maximum-Flow and Minimum-Cut Relationship

A cut partitions vertices into a source-side set and a sink-side set. Its capacity is the sum of capacities of directed edges that cross from the source side to the sink side. The max-flow min-cut theorem states that the maximum feasible flow equals the capacity of a minimum cut [2]. This relationship is valuable because it gives both a numeric answer and an explanation of the limiting boundary.

After no augmenting path remains in the residual graph, the vertices reachable from the source define the source side of a minimum cut. Original edges leaving that set toward unreachable vertices form the cut returned by the solver. The dashboard can therefore display which links constrain throughput for the current graph.

2.4 Edmonds–Karp Algorithm

Edmonds–Karp is a maximum-flow algorithm that uses breadth-first search to find augmenting paths in the residual graph [1]. The residual graph tracks unused forward capacity and the ability to cancel or revise previously assigned flow through reverse residual edges. Initially, each forward edge has its original capacity and each reverse edge has zero residual capacity. For each iteration, breadth-first search records a parent for each reached vertex. If the sink is reached, the algorithm traces parent links backward to find the minimum residual capacity on the path. This bottleneck amount is added to the total flow, subtracted from each forward residual edge, and added to each reverse residual edge. The process repeats until the sink cannot be reached.

Using breadth-first search makes the Edmonds–Karp procedure easier to analyze and implement than an arbitrary augmenting-path strategy. Its standard time complexity is O(VEZ) [1], [3]. For the small graph examples used in the dashboard, this method is understandable and sufficient. Very large or frequently changing production networks may require other algorithms or incremental techniques.

2.5 Dynamic Programming Foundations

Dynamic programming solves a problem by combining solutions to smaller subproblems [7]. In the 0/1 knapsack formulation, each candidate flow has a demand d_i and a value v_i. The capacity budget is C. The decision is binary: either a flow is selected once or it is not selected. The objective is to maximize the sum of selected values while keeping total demand at or below C.

Let DP[i][c] denote the maximum value achievable using the first i flows with capacity c. The recurrence compares excluding the current flow with including it when its demand fits:

$DP[i][c] = DP[i-1][c]$ if $d_i > c$; otherwise $DP[i][c] = \max(DP[i-1][c], DP[i-1][c-d_i] + v_i)$. The table is filled row by row.

The implementation then backtracks through the table to reconstruct the selected flow names. Its time complexity is $O(nC)$ and its memory complexity is $O(nC)$, where n is the number of flows and C is the integer capacity budget [5], [6]. This is pseudo-polynomial: performance depends on the numeric capacity, not only on the number of requests.

2.6 Serial and Parallel Computation

Serial execution solves scenarios one after another. Parallel execution distributes independent scenarios across worker processes, allowing multiple CPU cores to work concurrently. Because each benchmark scenario is initialized from a fixed seed, the generated input remains consistent regardless of the order in which scenarios are executed.

The benchmark measures wall-clock time using a high-resolution performance counter. Speedup is calculated as serial time divided by parallel time. A speedup greater than one indicates that the parallel run completed faster for that experiment; a value below one indicates that process management or other overhead outweighed the benefit. Results should be interpreted as measurements, not guarantees.

A multiprocessing pool introduces costs for process startup, task distribution, serialization, scheduling, and result collection [14]. Therefore, small workloads may be faster serially. The useful workload size depends on the processor, Python runtime, operating system, and complexity of each scenario. Repeated trials and documented configuration improve the reliability of comparisons.

2.7 Comparison of Related Approaches

Approach	Strength	Limitation	Role in this project
Greedy allocation	Simple and fast	May miss the best combination	Baseline concept
0/1 knapsack DP	Optimal for stated discrete model	Capacity-indexed memory and runtime	Implemented allocation
Edmonds–Karp	Clear maximum-flow/min-cut explanation	Can be expensive on large graphs	Implemented routing
Serial benchmark	Low coordination overhead	Does not use multiple workers	Reference timing
Multiprocessing	Can speed up independent workloads	Process overhead may dominate	Parallel timing

Table 2.1 Comparison of related approaches

The methods solve different parts of the problem and should not be treated as interchangeable. Maximum flow optimizes routing throughput over a graph, while knapsack allocation chooses a subset of requests under one capacity budget. Benchmarking compares execution strategies for repeated allocation scenarios; it does not alter the mathematical optimum.

2.8 Summary

The theoretical basis of the system is the combination of flow conservation, residual-network augmentation, the max-flow min-cut relationship, dynamic-programming optimal substructure, and independent workload parallelism. Each concept maps to a distinct module and produces a result that can be checked: total flow and cut edges, maximum utility and selected requests, or timing and output-equivalence measurements.

The project intentionally keeps its model small enough to explain. It does not model every dimension of operational traffic engineering, such as time-varying demand, multiple simultaneous capacity constraints, stochastic failure, latency-sensitive routing, or fairness across customers. These limitations guide the future scope discussed later in the report.

3 PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE

3.1 Introduction

The proposed methodology divides the application into three algorithm modules and one presentation/service layer. The maximum-flow module accepts directed edges and returns total throughput, edge flows, and minimum-cut edges. The allocation module accepts named traffic demands, utility values, and a capacity budget. The benchmark module generates seeded scenarios and compares serial and multiprocessing execution. Flask provides the API routes, while the dashboard collects user inputs and presents results.

This modular design allows each algorithm to be tested independently and reduces coupling between computation and display. The dashboard is a client of the API: it sends structured JSON, receives a result object, and updates the relevant panel. The same endpoints can be exercised using an API client, which makes it possible to validate server behavior separately from the interface.

3.2 System Requirements

3.2.1 Hardware Requirements

- **Processor:** a modern desktop or laptop CPU; multiple cores are useful for the multiprocessing benchmark.
- **Memory:** sufficient RAM for Python, Flask, browser tabs, and dynamic-programming tables.
- **Storage:** space for the Python source files, HTML templates, dependencies, and report artifacts.
- **Display:** a screen capable of presenting the dashboard panels and network graph.
- **Input:** keyboard and pointing device for editing capacities and running experiments.

3.2.2 Software Requirements

Component	Technology	Purpose
Programming language	Python 3	Implements optimization algorithms
Web framework	Flask	Serves the dashboard and JSON APIs
Graph algorithm	Edmonds–Karp	Computes maximum flow and minimum cut
Allocation algorithm	Dynamic programming	Selects requests under capacity
Parallel execution	multiprocessing	Evaluates scenarios across worker processes
Frontend	HTML, CSS, JavaScript	Provides input forms and result visualization
Testing	pytest or unittest	Checks solver behavior and edge cases

Table 3.1 Software requirements

3.3 Functional Requirements

Functional requirements describe the operations that the system must provide.

FR-1 — Maximum-flow calculation: the user can enter directed links and capacities and request a maximum-flow computation from S to T.

FR-2 — Edge-flow output: the response includes the computed flow on each original edge, so the total can be interpreted at link level.

FR-3 — Minimum-cut identification: the system returns the original edges crossing the source-side/sink-side partition in the final residual graph.

FR-4 — Bandwidth allocation: the user can enter flow names, demands, values, and a capacity budget; the system returns the best value and selected names.

FR-5 — Parallel benchmark: the user can specify a scenario count and view serial time, parallel time, worker count, speedup, and result-match status.

FR-6 — Dashboard integration: the user can switch among routing, allocation, and benchmark views without directly invoking solver functions.

3.4 Non-Functional Requirements

Performance: algorithms should complete in reasonable time for the documented demonstration inputs. Dynamic programming can consume substantial memory if the capacity value is very large, so production-quality code should enforce practical limits.

Correctness: maximum flow must respect capacity constraints and conservation. Allocation must not exceed the capacity budget, and the selected flows must achieve the reported value. Serial and parallel benchmark results should match for identical deterministic scenarios.

Usability: labels and examples should explain the expected input format. Errors should be reported clearly instead of leaving the user uncertain whether a request was processed.

Maintainability: solver functions are separated from Flask route handlers. This makes algorithms reusable from tests and easier to revise independently.

Reliability and safety: malformed JSON, missing fields, invalid numbers, negative capacities, and extreme workload requests should be validated before expensive computation. The prototype should not be exposed as an unrestricted public service without additional safeguards.

3.5 System Architecture

The architecture has four logical layers. The presentation layer contains the browser dashboard and forms. The API layer consists of Flask routes that parse requests and serialize results. The algorithm layer contains the maximum-flow, allocation, and benchmark functions. The runtime layer supplies Python execution and multiprocessing workers.

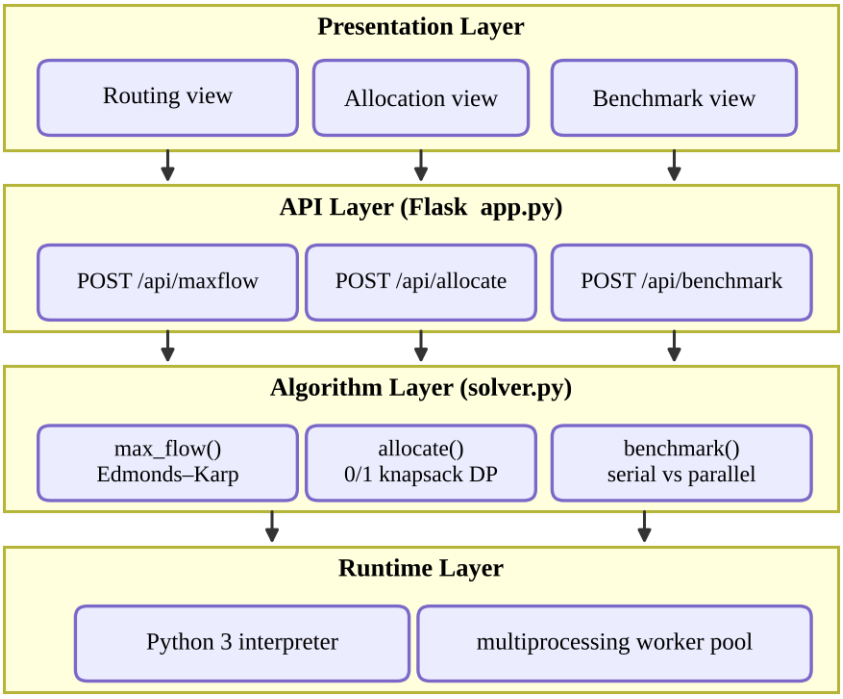


Figure 3.1: Logical architecture of the bandwidth allocation system

Layer	Components	Responsibility
Presentation	Dashboard HTML/CSS/JavaScript	Collect inputs and display results
API	Flask app.py	Validate request structure and route calls
Algorithm	solver.py	Compute flow, allocation, and benchmark results
Runtime	Python and multiprocessing	Execute calculations and worker tasks

Table 3.2 System architecture layers

The separation supports independent testing. The solver can be called directly by unit tests, the API can be tested using JSON requests, and the dashboard can be checked through browser interaction. In a production deployment, a production WSGI server, logging, configuration management, and request limits should be added.

3.6 Data Flow

For maximum-flow routing, the user enters rows containing a source vertex, destination vertex, and capacity. The frontend packages the rows into an edges array and posts the JSON object to /api/maxflow. Flask converts the capacity values to numbers and invokes max_flow. The response returns total, flows, and cut, which the interface renders as a throughput summary and graph annotations.

For allocation, the frontend sends each flow as a name-demand-value tuple and includes the capacity budget. Flask converts demand and value to integers and calls allocate. The solver fills the DP table and backtracks to produce the selected flow names. The response contains best and allocated, which the dashboard uses to show total utility and accepted/rejected statuses.

For benchmarking, the frontend sends a scenario count to /api/benchmark. The benchmark evaluates the same seed range serially and with a multiprocessing pool. The resulting JSON contains scenario count, worker count, serial time, parallel time, speedup, and result equality.

3.7 Maximum-Flow Workflow

The max-flow procedure first builds residual-capacity maps from the input edge list. Repeated edges between the same ordered pair are combined for capacity calculation. For every original edge, the solver records the original capacity so that the final flow can be calculated from the difference between original and remaining forward residual capacity.

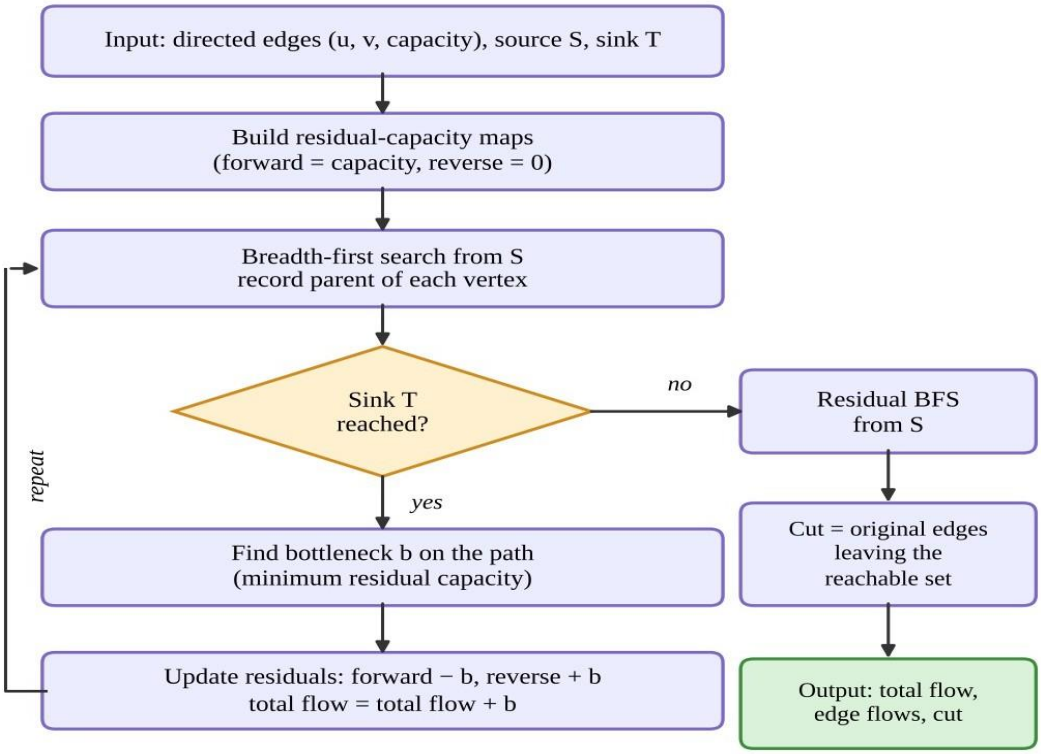


Figure 3.2: Maximum-flow computation workflow

The algorithm then performs breadth-first search from source S. If T is reached, parent pointers identify an augmenting path. The path bottleneck is the smallest residual capacity along that path. Updating forward and reverse residual capacities increases the total feasible flow while allowing later paths to revise earlier assignments.

When no augmenting path exists, a final residual traversal identifies all vertices reachable from S. Original edges leaving this reachable set toward an unreachable vertex form the minimum cut. The algorithm returns the total, a dictionary of edge flows, and the cut edge list.

3.8 Dynamic-Programming Allocation Workflow

The allocation function receives a list of flows, where each entry includes a name, demand, and value. It allocates a table with one row for each prefix of the flow list and one column for every capacity from zero through the budget. Each cell stores the maximum value achievable for that prefix and capacity.

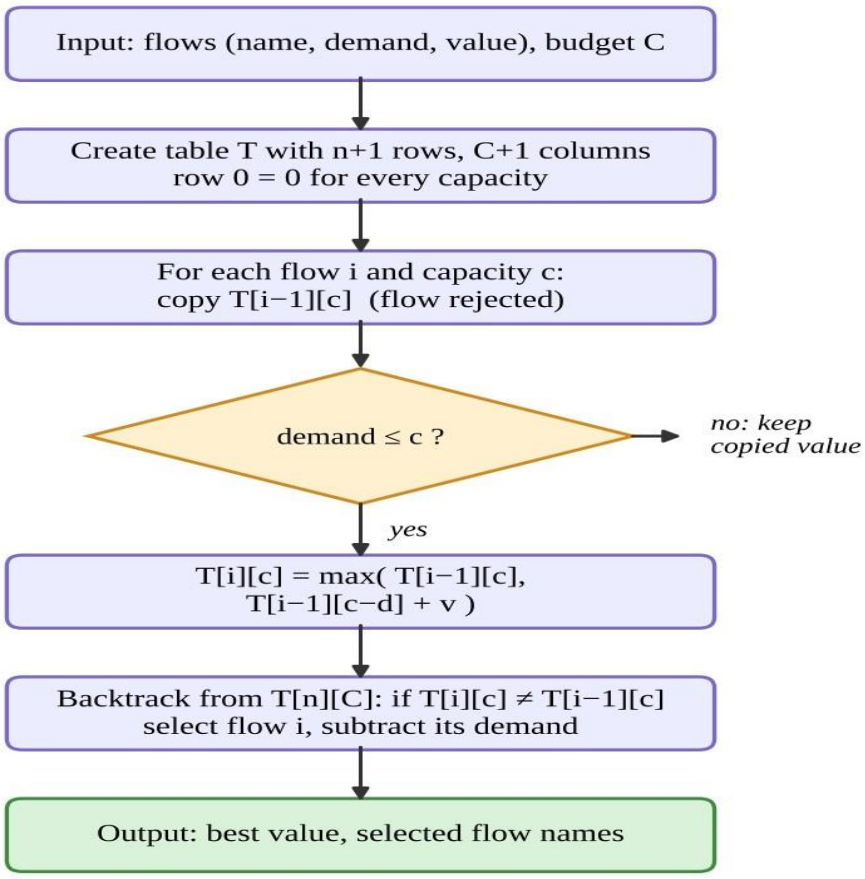


Figure 3.3: Dynamic-programming allocation workflow

For every flow, the solver copies the value from the previous row to represent rejection. If the flow fits, it also considers the previous-row value at the remaining capacity plus the current value. The larger result is stored. Because the recurrence uses the previous row for inclusion, each flow can be selected at most once.

Backtracking begins at the final table cell. When the value differs from the cell above it, the current flow is selected and its demand is subtracted from the remaining capacity. The selected names are reversed at the end to preserve input order. The resulting set is optimal for the supplied discrete 0/1 model.

3.9 Parallel Benchmark Workflow

Each benchmark scenario is generated from a deterministic integer seed. The scenario generator creates a fixed-size list of synthetic flows with demands and values drawn from a seeded random generator. The allocation solver returns the optimum value for the scenario. Using the same seed sequence in both modes ensures that the benchmark compares equivalent work. The serial section calls the scenario function for every seed in sequence and records elapsed time. The parallel section creates a multiprocessing pool using the detected

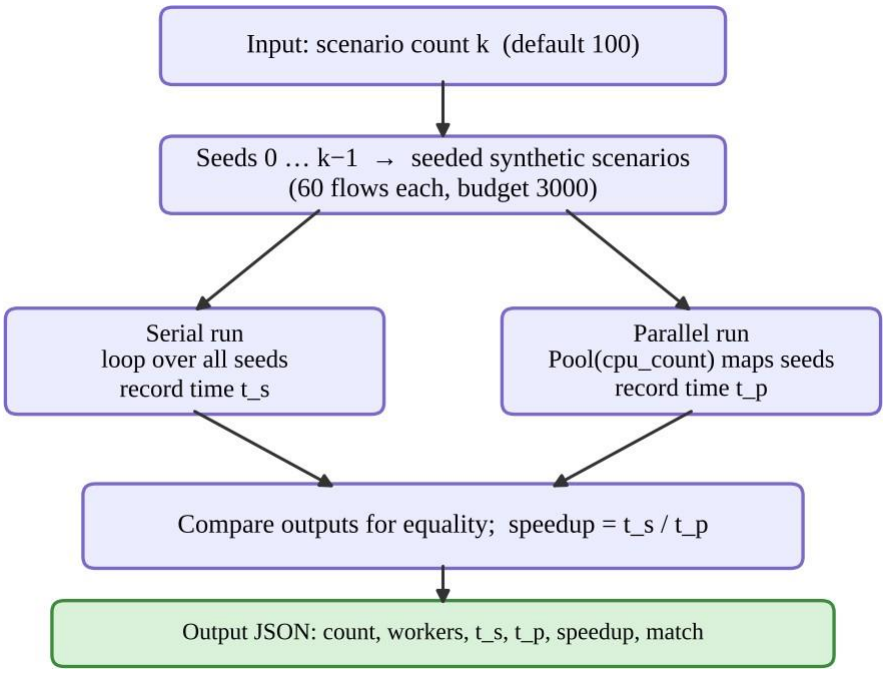


Figure 3.4: Serial and parallel benchmark workflow

The benchmark is a measurement of the complete execution path, including pool setup and result collection. It therefore reflects both computation and orchestration overhead. Repeating the test and comparing different scenario counts is recommended before drawing conclusions about scaling.

3.10 Complexity Analysis

Module	Time complexity	Space consideration
Edmonds–Karp	O(VEZ)	Residual graph and parent map
Knapsack DP	O(nC)	O(nC) table in this implementation
Serial benchmark	O(k × allocation cost)	One result per scenario
Parallel benchmark	Work divided across workers plus overhead	Worker processes and result collection

Table 3.3 Complexity of the implemented modules

V is the number of vertices, E is the number of edges, n is the number of candidate flows, C is the integer capacity budget, and k is the number of benchmark scenarios. The complexity expressions are standard theoretical descriptions; actual runtime depends on input size, implementation details, and the execution environment.

The dynamic-programming table is especially sensitive to the capacity value. A budget of 80 is small, but a capacity of several million could create an impractically large table. Input validation should therefore include a maximum supported budget or use a different formulation for large numerical capacities.

4 IMPLEMENTATION

4.1 Introduction

The application is implemented in Python using Flask as the web framework [13]. The app module defines the home route and three POST endpoints. The solver module provides the maximum-flow algorithm, dynamic-programming allocation, and benchmark routine. The dashboard template presents the three user-facing functions as tabs or panels.

The implementation keeps route handling relatively small: request data is converted to the expected types, passed to a solver function, and returned as JSON. This organization is useful for a project report because the core algorithms can be explained independently from HTTP request handling and interface presentation.

4.2 Project File Organization

File or directory	Purpose
app.py	Creates Flask application and API routes
solver.py	Implements max_flow, allocate, and benchmark
templates/index.html	Renders the browser dashboard
static/ (if used)	Stores CSS, JavaScript, and static assets
tests/ (recommended)	Contains solver and API tests
requirements.txt	Records required Python packages

Table 4.1 Project file organization

The supplied Flask code imports jsonify, request, and render_template. The home route returns the index template. The three POST routes accept JSON bodies and call the corresponding solver function. The server starts on port 5000 when the module is run directly, with Flask debug mode disabled.

For reproducibility, the project should include dependency versions, instructions for starting the application, and sample inputs. Separating source code from templates also makes the user interface easier to revise without changing the solver implementation.

4.3 Maximum-Flow Implementation

The max_flow function uses dictionaries to represent residual capacities. For each edge (u, v, c), it ensures both vertices have adjacency maps, adds capacity c to the forward direction, and initializes the reverse direction if needed. The original capacity is stored separately so that the final flow for each original edge can be reported.

Breadth-first search uses a deque. The parent map doubles as a visited set, and the search stops early once the sink has been discovered. If the sink is not present in the parent map, no augmenting path exists and the flow loop terminates.

The bottleneck is calculated by walking backward from sink to source through parent links. A second walk updates residual capacities. The total is increased by the bottleneck amount. After termination, a reachability search through positive residual capacities identifies the source side of the cut.

4.4 Edge Cases and Validation

A disconnected sink should produce zero maximum flow. A single path is limited by its smallest edge capacity. Parallel input edges can be aggregated when they share the same direction. A directed reverse edge supplied by the user is distinct from the residual reverse edge created by the algorithm.

The current route converts capacities to floating-point values. A robust version should reject negative, infinite, or NaN capacities, ensure that every edge has three fields, and provide meaningful errors for malformed JSON. If the graph does not contain source S or sink T, the implementation should handle the condition predictably rather than assuming every label exists in an adjacency map.

A useful verification is to check flow conservation at intermediate vertices and confirm that the capacity of the returned cut equals the reported maximum flow. These properties can detect errors that may not be visible from the total alone.

4.5 Dynamic-Programming Allocation Implementation

The allocate function creates a table T with n+1 rows and capacity+1 columns. The first row represents the empty set of candidate flows and therefore has value zero for every capacity. Each subsequent row adds one flow to the available decision set.

For a current flow with demand d and value v, the function first copies the best value from the preceding row. If the current capacity c is at least d, the function compares the copied value with T[i−1][c−d] + v. The larger value becomes T[i][c]. This recurrence ensures the current flow is not reused multiple times.

After the table is complete, the algorithm walks backward through its rows. If T[i][c] differs from T[i−1][c], the flow at i−1 is part of the selected solution. Its name is appended and its demand is subtracted from c. The final selected list is reversed before returning the best value and chosen names.

4.6 Worked Allocation Example

Consider a capacity budget of 10 units and three candidate flows: Video with demand 6 and value 12, Voice with demand 3 and value 7, and Backup with demand 5 and value 8. Video and Voice together consume 9 units and produce value 19. Voice and Backup consume 8 units and produce value 15. Video and Backup require 11 units and cannot both be accepted.

Flow	Demand	Value	Decision	Reason
Video	6	12	Allocated	Fits within the selected combination
Voice	3	7	Allocated	Adds value 7 using 3 units
Backup	5	8	Rejected	Would not improve the best feasible subset

Table 4.2 Worked allocation example (budget = 10 units)

For this example, the optimal total value is 19. The selected demand is 9, so one unit remains unused. The unused capacity does not indicate a solver failure; it means no available combination of the remaining flows improves the objective while satisfying the budget.

4.7 Flask API Design

Endpoint	Method	Input	Response
/	GET	No body	HTML dashboard
/api/maxflow	POST	edges array	total, flows, cut
/api/allocate	POST	flows and capacity	best, allocated
/api/benchmark	POST	optional count	timings, speedup, match

Table 4.3 API endpoints and request fields

The /api/maxflow route reads request.json['edges'] and converts each capacity to a float. It invokes max_flow and returns a JSON object containing total, flows, and cut. The /api/allocate route reads flow tuples, converts demand and value to integers, converts the budget to an integer, and returns best and allocated. The benchmark route uses a default count of 100 if the request does not supply count.

The routes are easy to exercise with browser developer tools or an API client. Before wider deployment, request schemas, bounds, exception handling, content-type checks, and HTTP status codes should be defined explicitly.

4.8 Sample API Requests

Maximum-flow request

```
POST /api/maxflow
Content-Type: application/json
{"edges":[["S","A",10],["S","B",8],["A","T",10],["B","T",8]]}
```

This example contains two independent source-to-sink paths with capacities 10 and 8. The maximum throughput is 18 units because the path capacities can be used together without sharing a smaller downstream bottleneck.

Allocation request

```
POST /api/allocate
Content-Type: application/json
{"flows":[["Video",6,12],["Voice",3,7],["Backup",5,8]],"capacity":10}
```

The expected optimum is 19, obtained by selecting Video and Voice. These requests are illustrative verification inputs. The actual response should be captured from the running application when recording final experimental evidence.

From	To	Capacity (Mbps)
S	A	10
S	B	8
A	T	10
B	T	8

Table 4.4 Sample network links and capacities

4.9 Benchmark Implementation

The scenario helper creates a seeded random generator and generates 60 candidate flows with demands and values inside configured ranges. It calls allocate with a capacity budget of 3000 and returns the optimum value for the scenario. The seed is passed as an argument, so the same seed produces the same scenario under the same implementation.

The benchmark measures a serial list comprehension over the requested seed range. It then creates a multiprocessing Pool using cpu_count() and maps the same helper over the same seed range. The output values are compared to determine whether the two modes produced identical results.

The returned JSON includes the scenario count, worker count, serial time, parallel time, speedup, and result-match status. Because pool creation and inter-process coordination are included in the timing, the result represents end-to-end benchmark duration rather than only the time spent inside the allocation recurrence.

4.10 Dashboard Interface

The supplied dashboard contains three main views. The Max-Flow Routing view accepts network links and displays throughput, bottleneck information, and a network diagram with edge labels. The DP Flow Allocation view accepts named demands, values, and a capacity budget, then presents accepted and rejected flows. The Parallel Benchmark view accepts a scenario count and displays serial time, multiprocessing time, speedup, and output equality.

The interface uses the same project title and visual style across the three views. Separating functions into tabs reduces clutter and helps the user focus on one algorithm at a time. The graph view makes capacity constraints easier to interpret, while the allocation table shows the relationship between demand, value, and admission decision.

The screenshots provided with the project serve as example interface evidence. They show a maximum throughput of 18 Mbps, an allocation value of 240 using an 80 Mbps budget, and a sample benchmark using 400 scenarios with a reported 1.59× speedup. These are screenshot observations and should not be generalized to other machines or inputs.

5 EXPERIMENTAL SETUP, RESULTS AND DISCUSSION

5.1 Experimental Setup

The system is evaluated using synthetic inputs and dashboard examples. Synthetic data makes it possible to repeat test scenarios without requiring access to a production network or private traffic traces. The graph input consists of directed edges and capacities. The allocation input consists of flow names, integer demands, integer values, and a capacity budget. Benchmark scenarios are generated from deterministic seeds.

The experimental process has three parts. First, maximum-flow examples are used to inspect the total flow and minimum-cut output. Second, allocation examples are used to confirm that the chosen demands fit the budget and achieve the reported total value. Third, benchmark scenarios are evaluated serially and in parallel, with the outputs compared for equality.

The sample values reported below are drawn from the dashboard screenshots supplied with the project. They should be treated as observed demonstration values, not as independently repeated laboratory measurements. A final submission should include the machine configuration and test outputs from the version actually executed.

5.2 Test Environment and Parameters

Parameter	Value or configuration	Notes
Application framework	Flask	Serves dashboard and JSON endpoints
Maximum-flow algorithm	Edmonds–Karp	Source S, sink T
Allocation algorithm	0/1 knapsack DP	Integer demands and capacity
Benchmark scenarios	400 in screenshot	Seeded synthetic scenarios
Workers	8 in screenshot	Machine-specific multiprocessing count
Example bandwidth budget	80 Mbps	Shown in allocation dashboard
Example max throughput	18 Mbps	Shown in routing dashboard
Example allocation value	240	Utility score, not necessarily monetary

Table 5.1 Test environment and parameters

For repeatable evaluation, the same inputs should be used for repeated trials and the Python version, operating system, CPU, memory, and worker count should be recorded. Timings may vary with background processes and machine load.

5.3 Maximum-Flow Results

The routing dashboard example reports maximum throughput of 18 Mbps and identifies S→A and S→B as the minimum-cut edges. The capacities of these source links are 10 Mbps and 8 Mbps, so their combined capacity is 18 Mbps. This agrees with the reported total and provides an intuitive explanation of the bottleneck in the displayed graph.

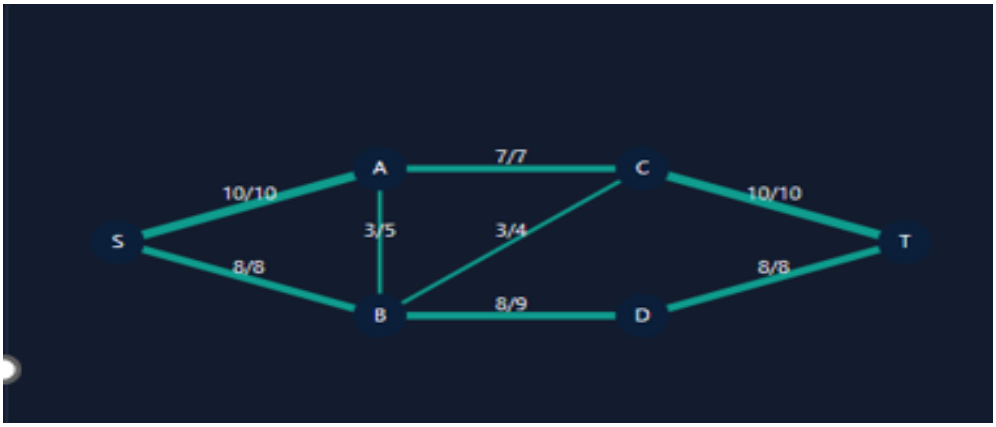


Figure 5.1: Max-flow routing dashboard

Metric	Dashboard observation	Interpretation
Maximum throughput	18 Mbps	Largest feasible source-to-sink flow for the shown input
Cut edge 1	S→A	Source-side boundary edge with capacity 10
Cut edge 2	S→B	Source-side boundary edge with capacity 8
Cut capacity	18 Mbps	Matches the displayed maximum-flow value

Table 5.2 Maximum-flow dashboard observations

The result illustrates why the minimum cut is useful: it identifies the set of links whose combined capacity limits the total flow. In a practical planning exercise, a capacity upgrade would require recalculating the graph because another edge may become the new bottleneck.

5.4 Dynamic-Programming Allocation Results

The allocation dashboard example uses a bandwidth budget of 80 Mbps and reports a best total value of 240. The value is a utility score assigned to each candidate flow; it is not necessarily a monetary quantity. The dashboard lists which flows were accepted and which were rejected, so the user can verify that the demands of the accepted flows add up to no more than the budget.

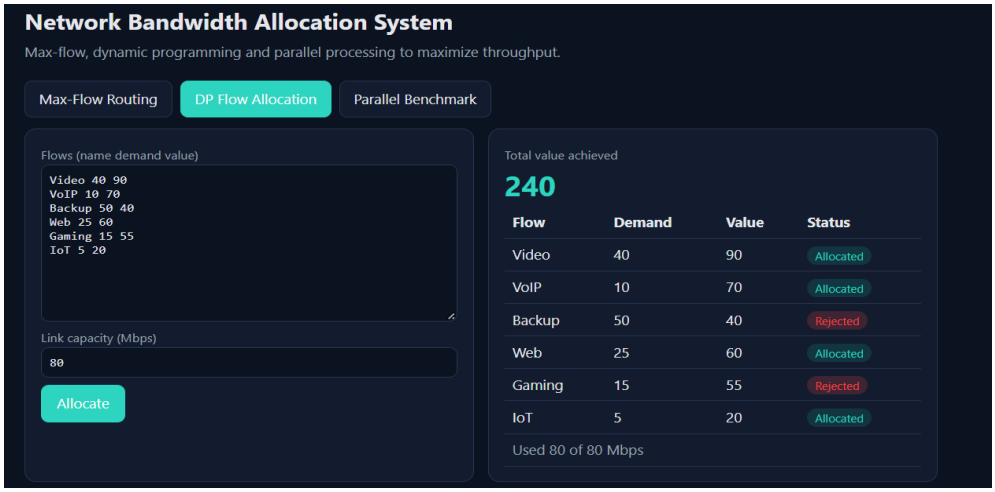


Figure 5.2: DP flow allocation dashboard

Item	Dashboard observation	Interpretation
Bandwidth budget	80 Mbps	Upper limit on the total demand of accepted flows
Best total value	240	Maximum utility reported for the entered flows
Accepted flows	Listed by the dashboard	Their demands must sum to at most 80 Mbps
Rejected flows	Listed by the dashboard	Not part of the optimal subset for this budget

Table 5.3 Allocation dashboard observations

The small worked example in Section 4.6 can be checked by hand and shows the same behaviour on a case that fits on one line: the optimum of 19 is obtained by Video and Voice, leaving one unit of the budget unused. A greedy policy that accepted flows in arrival order, or by highest value alone, could consume capacity on a single large flow and miss a better

combination. The dynamic-programming table examines every feasible capacity state and therefore does not have this weakness for the stated 0/1 model.

Two properties should be confirmed for every allocation result: the sum of the accepted demands must not exceed the budget, and the sum of the accepted values must equal the reported best value. Both can be checked directly from the dashboard output without re-running the solver.

5.5 Parallel Benchmark Results

The benchmark dashboard example evaluates 400 seeded scenarios with 8 worker processes and reports a speedup of 1.59×. Because speedup is defined as serial time divided by parallel time, the value means that the multiprocessing run finished about 1.59 times faster than the serial run for that experiment.

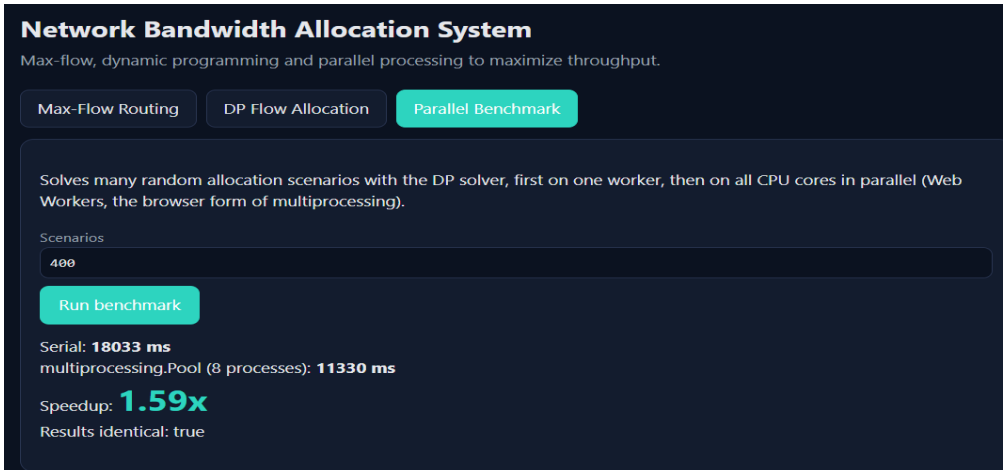


Figure 5.3: Parallel benchmark dashboard

Item	Dashboard observation	Interpretation
Scenarios	400	Seeded synthetic allocation problems
Workers	8	Process count detected on the test machine
Speedup	1.59×	Serial time divided by parallel time
Parallel efficiency	≈ 20%	1.59 ÷ 8 workers; computed from the two values above

Table 5.4 Parallel benchmark dashboard observations

A speedup of 1.59× on eight workers corresponds to a parallel efficiency of roughly 20%. This is expected for a workload of this size: each scenario is small, and the measured parallel time includes creating the process pool, distributing the seeds, serializing the results and collecting them. The result therefore supports the statement in Section 2.6 that parallel execution pays off only when the work per task is large enough to outweigh process overhead.

The reported value is a single observation on one machine. It should not be generalized to other processors, operating systems or scenario counts. Repeating the benchmark with several

scenario counts, and recording the Python version and processor, would show how the speedup changes as the workload grows.

5.6 Test Plan and Expected Observations

The test plan below lists the cases that verify the three modules. The expected observations follow directly from the algorithm descriptions in Chapters 2 to 4. They are expectations to be confirmed by running the application; they are not reported as completed measurements.

ID	Test case	Input	Expected observation
T1	Two independent paths	Edges S→A 10, S→B 8, A→T 10, B→T 8	Maximum flow 18; cut edges S→A and S→B
T2	Disconnected sink	Graph with no path from S to T	Maximum flow 0; empty flow on every edge
T3	Single path	S→A 10, A→B 4, B→T 9	Maximum flow 4 (smallest edge capacity)
T4	Parallel input edges	Two S→A edges with capacities 3 and 5	Capacity treated as 8 for the S→A direction
T5	Worked allocation	Video 6/12, Voice 3/7, Backup 5/8; budget 10	Best value 19; Video and Voice selected
T6	Zero budget	Any flows; budget 0	Best value 0; no flow selected
T7	Oversized demand	Flow whose demand exceeds the budget	Flow is never selected
T8	Seed repeatability	Same scenario count run twice	Identical optimum values in both runs
T9	Serial/parallel equality	400 scenarios, serial then parallel	Output lists match; speedup reported

Table 5.5 Test plan and expected observations

Cases T1 and T5 are reproduced by the two sample API requests in Section 4.8. Cases T2 to T4 and T6 to T7 exercise edge conditions that are easy to get wrong in graph and table code. Cases T8 and T9 confirm that the deterministic seeding makes the serial and parallel benchmarks comparable.

5.7 Discussion

The three modules answer three different questions and should be read together. The maximum-flow module shows how much traffic a given topology can carry and which links form the bottleneck; in the dashboard example the two source links, with capacities 10 and 8, form the minimum cut and explain the throughput of 18 Mbps. The allocation module shows which demands should be admitted when a single budget cannot satisfy all of them; in the dashboard example a budget of 80 Mbps yields a best value of 240. The benchmark module shows what the computation costs and whether parallel execution helps.

The observations are consistent with the theory in Chapter 2. The cut capacity equals the maximum flow, as the max-flow min-cut theorem requires. The allocation result respects the budget and is optimal for the discrete model. The benchmark speedup is below the number of workers, as expected when process overhead is included in the measurement.

Several limits apply. The inputs are synthetic or hand-entered, so the results illustrate the methods rather than describe a production network. Capacities and utilities are treated as given data. The dynamic-programming table grows with the numeric budget, and the benchmark result depends on the test machine. These limits are discussed further in Chapter 6.

6 CONCLUSION, LIMITATIONS AND FUTURE SCOPE

6.1 Conclusion

This project developed a Network Bandwidth Allocation System that combines maximum-flow routing, dynamic-programming bandwidth allocation and a serial/parallel benchmark behind a Flask service and a browser dashboard. The Edmonds–Karp module computes the largest feasible flow from S to T, reports the flow on every original edge and identifies the minimum-cut edges that explain the bottleneck. The knapsack module selects the subset of candidate flows with the highest total value that fits the bandwidth budget, and reconstructs the chosen flows by backtracking through the table.

The benchmark module evaluates seeded synthetic scenarios serially and with a multiprocessing pool and reports elapsed time, worker count, speedup and result equality together. In the supplied dashboard examples the routing view reports a maximum throughput of 18 Mbps with a cut of 18 Mbps, the allocation view reports a value of 240 under an 80 Mbps budget, and the benchmark reports a speedup of 1.59× over 400 scenarios.

The project achieves its objective of making the three algorithms visible, testable and repeatable. Keeping the modules separate allows each one to be understood and tested on its own, and exposing them through JSON endpoints allows the same computations to be used from the dashboard or from any API client.

6.2 Limitations

- The model is a simplified single-commodity flow with one source and one sink; it does not represent several simultaneous traffic classes.
- Allocation uses a single bandwidth budget and integer demands; the table size grows with the numeric budget.
- Capacities and utility scores are supplied as input data. The system does not collect live telemetry, measure delay or loss, or reserve bandwidth on real devices.
- The current routes convert capacities with minimal validation; negative, infinite or malformed values should be rejected before computation.
- Benchmark results are machine-specific single observations and are not a controlled scaling study.
- The prototype uses the Flask development server and has no authentication, request limits or logging, so it should not be exposed as a public service.

6.3 Future Scope

- Add request schemas, bounds on the budget and graph size, and explicit HTTP error codes.

- Add automated unit tests for the solver and API tests for the three endpoints, including the edge cases in Table 5.5.
- Compare Edmonds–Karp with faster maximum-flow algorithms such as Dinic's algorithm or push-relabel on larger graphs [9], [10].
- Extend allocation with priorities, fairness constraints, multiple budgets or minimum-cost flow.
- Import live link measurements and visualize utilization over time.
- Repeat the benchmark across scenario counts and machines and report confidence intervals.
- Deploy behind a production WSGI server with logging, configuration management and request limits.

7 REFERENCES

[1] J. Edmonds and R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems," Journal of the ACM, vol. 19, no. 2, pp. 248–264, 1972.

[2] L. R. Ford Jr. and D. R. Fulkerson, "Maximal flow through a network," Canadian Journal of Mathematics, vol. 8, pp. 399–404, 1956.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[4] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, Network Flows: Theory, Algorithms, and Applications. Englewood Cliffs, NJ, USA: Prentice Hall, 1993.

[5] S. Martello and P. Toth, Knapsack Problems: Algorithms and Computer Implementations. Chichester, U.K.: Wiley, 1990.

[6] H. Kellerer, U. Pferschy, and D. Pisinger, Knapsack Problems. Berlin, Germany: Springer, 2004.

[7] R. Bellman, Dynamic Programming. Princeton, NJ, USA: Princeton University Press, 1957.

[8] J. Kleinberg and É. Tardos, Algorithm Design. Boston, MA, USA: Pearson Addison-Wesley, 2006.

[9] E. A. Dinic, "Algorithm for solution of a problem of maximum flow in networks with power estimation," Soviet Mathematics Doklady, vol. 11, pp. 1277–1280, 1970.

[10] A. V. Goldberg and R. E. Tarjan, "A new approach to the maximum-flow problem," Journal of the ACM, vol. 35, no. 4, pp. 921–940, 1988.

[11] D. Bertsekas and R. Gallager, Data Networks, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1992.

[12] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. Hoboken, NJ, USA: Pearson, 2021.

[13] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com/

[14] Python Software Foundation, "multiprocessing — Process-based parallelism," Python 3 documentation. [Online]. Available: https://docs.python.org/3/library/multiprocessing.html